Pipeline Optimization Summary

Shopify ETL pipeline (aws-pipeline-ec2) and its workers | October 2026

1. At a glance

- **Events data now takes one path.** Raw events are pulled from Mongo once, every 15 minutes, into MySQL; the other workers read the MySQL copy. The Mongo copy is deleted after it is safely stored (7,213 documents cleared in the first run alone).
- **The database is more reliable.** Connections are pooled, a silent process crash during backfills was fixed, and two silent data-loss issues were closed.
- **The high RDS read load was traced to one worker** (variant-wise events) that kept full-scanning three large tables. After the fix, average reads fell from about 2,150 to about 510 per second (roughly 76% lower).

2. The RDS read-load problem

What we saw: read IOPS of 1,500-2,800 (the instance baseline is about 2,000), disk queue depth of 10-30, and the I/O burst balance falling from 38% to 24%. Throughput was not the limit, so the cause was a large number of small reads, not heavy data volume.

What caused it: the variant-wise events worker reads two days of events from *mongo_events_summary* for every brand. MySQL scans the whole table to do this (TMC 4.0M rows / 5.3 GB, BBB 1.7M / 5.6 GB, AJMAL 2.2M / 3.8 GB), because the two-day window covers 28-41% of each table, so an index would not help. Each large brand took 2-3 minutes to scan. The worker was meant to run every 30 minutes, but passes were observed starting about every 9 minutes, back to back.

Measure	Before (19 Oct, 24-minute live sample)	After the fix (21 Oct, ~20 minutes of readings)
Variant-wise scan passes per day	About 160 (a new pass roughly every 9 minutes)	1 (last run 00:20-00:29 IST; none since)
Scan seen running when sampled	In all 47 of 47 half-minute samples	In 0 of 283 one-second samples (5 minutes)
Read IOPS	Average 2,153, range 920-3,000 (never below 920)	Average about 510, range 0-2,547 (about 76% lower)
Read throughput	Average about 35 MB/s, range 15-49 MB/s	Average about 8 MB/s, range 0-42 MB/s (about 76% lower)

The fix (small and safe): the worker now runs once a day at 00:15 IST instead of repeatedly, with no run at container start, and a lock stops two runs from overlapping. The query, the date window and all business logic are unchanged. The window is the two closed days before today, so re-running it more often never produced new data.

Checked and ruled out: the main pipeline's date queries (they use indexes and read 3.5k-20k rows), the intent pipeline (small tables, indexed), the returns table, and the events-summary ingest (healthy, on its 15-minute schedule).

3. Other optimizations

Issue	What changed	Before	After	Status
Four code paths each queried Mongo for the same events	Single ingest point (events-summary-worker); other workers read the MySQL copy	4 separate Mongo query paths	1 Mongo query path	**In production**
Mongo collection was never cleaned and kept growing	Documents are deleted from Mongo after they are stored in MySQL	Nothing ever deleted	7,213 documents cleared in the first run (7 brands)	**In production**
Ingest re-read a fixed 2-day window once a day	Reads only events newer than the last saved position, every 15 minutes	Whole 2-day window re-read daily	Only new events, every 15 minutes	**In production**
Every database call opened a new connection (full TCP, TLS and login each time)	Per-brand connection pools	About 13 new connections per brand per startup pass	Pool of 5 (+2 spare) reused per brand	**In production**
Pipeline silently exited during multi-brand backfills	Switched MySQL driver mode to the pure-Python one (the C version crashes under parallel SSL connects)	Process died with no error right after starting parallel brand threads	No native crash expected	**Built, awaiting prod check**
Decimal prices were saved as NULL (e.g. 299.5 into a whole-number column)	Column is widened automatically when a decimal value first appears	Cart price and discount decimals lost for TMC and AJMAL	Decimals stored; older NULLs are not back-filled	**Built, awaiting prod check**
Order rebuilds could reset event totals to 0 in overall_summary	Rebuild now carries the existing event totals forward	Totals could be wiped on each order-driven rebuild	Totals preserved	**In production**
Event data stayed stale after a restart	Events-summary-worker runs once immediately at startup	Up to 24 hours stale after a restart	Fresh immediately	**In production**
Last-order lookup scanned the whole orders table (no index on created_at)	Index idx_so_created_at added	BBB today: full scan of 1,051,152 rows, 31 s observed (AJMAL once timed out at 120 s)	TMC and AJMAL: 1 row read. Still missing on BBB, PTS, VAMA, 3TENX, SHYLENEW, URBAN HINDU	**Applied on 2 of 8 brands**
Intent-events worker no longer belongs in this deployment	Removed from compose and code; archived under dead_code/	6 services in this stack	5 services	**Code done; confirm old container is stopped**

Notes on the numbers: both measurements used read-only queries against the same database server, taken around late morning IST (19 Oct 11:09-11:43; 21 Oct 10:39-10:59). Short bursts of up to ~2,500 reads/s still occur after the fix, from other jobs, but the sustained load is gone. "In production" means seen working in production logs or database checks. Items marked pending have not been re-checked against a live run.